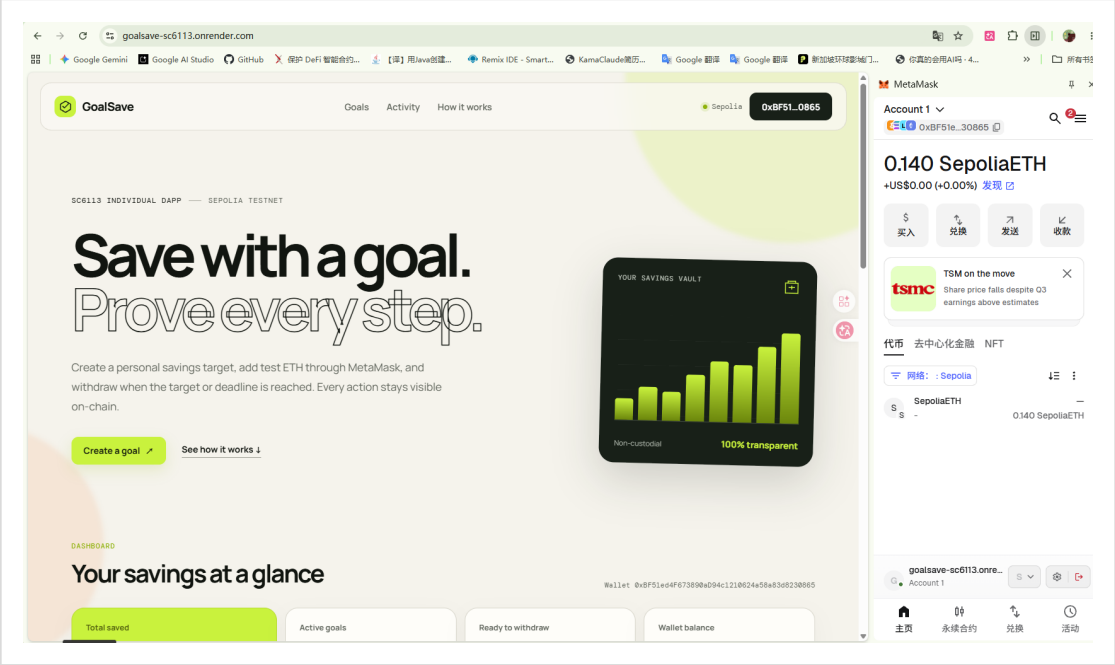
SC6113 BLOCKCHAIN APPLICATIONS

GoalSave Individual Assignment Report

Goal based savings DApp on Ethereum Sepolia



Live Render application connected to MetaMask on Sepolia

Student	Module	Submission
HE HUZHENGXIONG G2509011E	SC6113	Individual Assignment
Project	Network	Date
GoalSave	Ethereum Sepolia Chain ID 11155111	18 October 2026

GoalSave implements a complete DApp lifecycle: a Solidity contract stores savings goals and controls ETH withdrawals; MetaMask signs every state changing action; Flask serves configuration and validation; and the public Render interface reads contract state and events from Sepolia.

Key result The deployed application completed a live create, deposit, target reached, and withdrawal demonstration. Etherscan recorded the contract returning 0.002 Sepolia ETH to the connected wallet.

01 INTRODUCTION AND SCOPE

Financial problem and project objectives

GoalSave addresses a simple financial problem: a person can set a savings target, but a private spreadsheet or banking note does not provide a shared and independently verifiable history. The DApp records the goal rules, deposits, and final outcome on Sepolia.

Problem statement

A conventional savings note can be edited without leaving an audit trail. It also gives contributors no direct way to verify the current balance or confirm that the owner followed the stated withdrawal rule. GoalSave uses an Ethereum smart contract as the source of truth for ownership, target amount, deadline, balance, and status.

Objectives

- Allow one wallet to create multiple named savings goals with a target and future deadline.
- Accept real Sepolia ETH deposits and calculate progress from contract state.
- Permit normal withdrawal only after the target is reached or the deadline passes.
- Provide a separately labelled emergency close path and a permanent event trail.
- Give clear wallet, network, transaction, validation, and error feedback in the web interface.

Technologies used

Layer	Technology	Role in GoalSave
Smart contract	Solidity 0.8.30	Goal state, ETH accounting, access control, events
Blockchain	Ethereum Sepolia	Public testnet execution and transaction evidence
Wallet	MetaMask	Account connection, network switching, transaction signing
Frontend	HTML, CSS, JavaScript, Ethers.js	Responsive dashboard and contract interaction
Backend	Python Flask	Hosting, ABI/config API, input validation, health check
Hosting	Render	Public Gunicorn web service
Version control	GitHub	Source code and deployment configuration
Testing	Node test runner, Ganache, unittest	Contract and Flask verification

Assignment fit

The project is a digital savings and deposit system, one of the financial application areas identified in the individual assignment brief. It includes the required Solidity, ABI, Flask, HTML, CSS, JavaScript, GitHub, Render, transaction history, validation, and screenshots.

02 SYSTEM ARCHITECTURE

Wallet first design with a thin backend

The blockchain is the source of truth. Flask serves the web application and performs supporting validation, while MetaMask keeps the private key and all transaction approvals under the user's control.

BROWSER	METAMASK	SEPOLIA
HTML and CSS interface Ethers.js contract client Goal and event rendering	Account selection Sepolia network approval Transaction signature	GoalSave contract ETH custody Immutable event log
FLASK ON RENDER	**JSON ENDPOINTS**	**PUBLIC EXPLORER**
Template and static hosting Gunicorn process Health monitoring	ABI and chain config Goal validation Deposit validation	Sepolia Etherscan Deployment receipt Action receipts

Transaction flow

- The page retrieves the ABI, contract address, chain ID, deployment block, and explorer base URL from the Flask config endpoint.
- The user connects MetaMask. The application verifies chain ID 11155111 before enabling state changing actions.
- Flask validates goal and deposit form data before JavaScript builds the contract call.
- MetaMask presents the transaction. The user approves it without exposing the private key to Flask or Render.
- After confirmation, the page reloads goal state and contract events from the deployment block.
- Each activity row links to Sepolia Etherscan for independent verification.

Backend responsibilities

Endpoint	Purpose	Security behavior
/health	Render service monitoring	Returns status only
/api/config	ABI and public chain configuration	No private key or secret is returned
/api/validate/goal	Name, target, and deadline checks	Rejects blank, zero, excessive, or past values
/api/validate/deposit	ETH decimal validation	Rejects zero and more than 18 decimals
/api/transaction/	Optional receipt lookup	Validates the hash and handles missing RPC safely

No database is required because ownership, amounts, status, and history already exist on-chain. Avoiding a second writable store prevents conflicts between a local database and the smart contract.

03 SMART CONTRACT DESIGN

Savings rules enforced in Solidity

GoalSave.sol is a single contract with no external imports. It uses actual msg.value deposits, explicit lifecycle states, custom errors, and event records. The optimized deployed bytecode is 4,040 bytes.

Function	Who or value	Effect
createGoal	Any wallet, no ETH	Validates name, target, deadline; stores a new goal ID
deposit	Any wallet, msg.value > 0	Adds the received ETH to one active goal
withdraw	Goal owner only	Unlocks after target or deadline; closes as Withdrawn
emergencyWithdraw	Goal owner only	Returns the current balance; closes as EmergencyClosed
getGoal	Read only	Returns the stored goal structure
getGoalIds	Read only	Returns all goal IDs created by an owner

Lifecycle

State	Entry condition	Available next actions
Active	Goal created	Deposit, unlocked withdrawal, or emergency close
Withdrawn	Normal withdrawal succeeds	Read only terminal state
EmergencyClosed	Early close succeeds	Read only terminal state

Security and correctness decisions

- Authorization compares msg.sender with the stored owner on both withdrawal paths.
- The contract changes savedAmount and status before transferring ETH, following checks effects interactions.
- A re-entrancy guard prevents a second payout from entering during an external transfer.
- Direct receive and fallback transfers revert, so every accepted ETH deposit is linked to a goal ID.
- Normal withdrawal requires savedAmount to reach targetAmount or block.timestamp to reach the deadline.
- Normal withdrawal and emergency close emit different events, preserving their meaning in the history.

ETH accounting

The deposit function records msg.value, which is the amount the contract actually receives. It does not trust a separate amount supplied by the browser. This prevents the interface value from disagreeing with the on-chain balance.

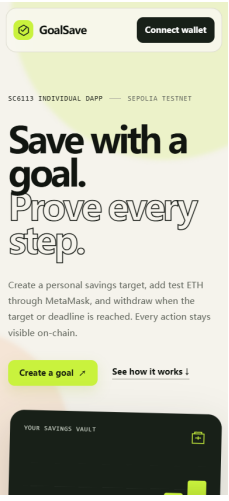
The browser disables Withdraw while the goal is locked, but the Solidity condition remains the authoritative control if a caller bypasses the interface.

04 APPLICATION DESIGN

Interface and backend implementation

The interface keeps the connected wallet and Sepolia network visible, then presents savings progress and the event trail in one page. Every state changing action provides pending, confirmed, or error feedback.

Interface area	Implementation
Header	Wallet connection, shortened account, Sepolia status
Dashboard	Total saved, active goals, ready goals, wallet balance
Goal cards	Name, balance, target, progress, deadline, lifecycle status
Actions	Create, deposit, conditional withdraw, emergency close
Activity	Contract events with amounts and Etherscan transaction links
Feedback	Validation messages, rejected signature, wrong network, RPC and contract errors



Responsive mobile layout

Implementation files

Required item	Submitted file	Purpose
Solidity	contracts/GoalSave.sol	Contract rules and events
ABI	abi/GoalSave.json	JavaScript contract interface
Python Flask	app.py	Web service, validation, public config
HTML	templates/index.html	Page structure and forms
CSS	static/styles.css	Responsive visual design
JavaScript	static/app.js	MetaMask, Ethers.js, UI state
Render	render.yaml and Procfile	Deployment and Gunicorn startup
Deployment	deployment.json	Sepolia address, block, transaction hash

Error handling

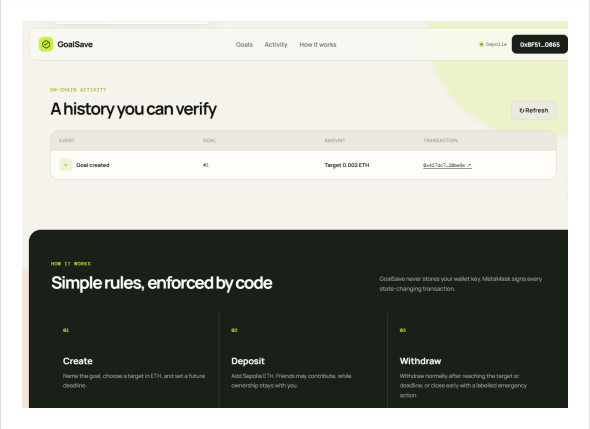
- The form rejects blank names, non-positive targets, expired deadlines, zero deposits, and excess decimal precision.
- The page detects missing MetaMask, incorrect networks, cancelled signatures, insufficient funds, and contract reverts. Render receives only public configuration; the seed phrase and private key never enter the repository or backend.

05 TESTING AND RESULTS

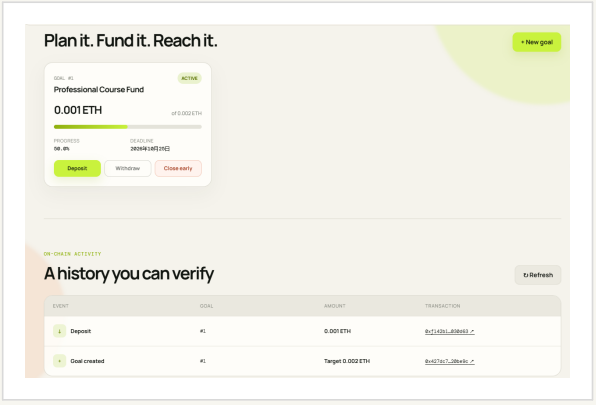
Automated checks and live Sepolia transactions

The final code passed eight smart contract tests and six Flask tests. JavaScript syntax validation also passed. The live demonstration then verified the complete user flow against the deployed Sepolia contract.

Layer	Result	Coverage
Smart contract	8 of 8 passed	Creation, deposits, invalid calls, authorization, locks, both withdrawal paths, direct transfer rejection
Flask	6 of 6 passed	Home, health, config, form validation, transaction hash and RPC fallback
JavaScript	Syntax check passed	static/app.js parsed successfully
Live Render	Operational	/health returned ok and /api/config returned the deployed Sepolia configuration



Goal number 1 created with a 0.002 ETH target



First 0.001 ETH deposit recorded at 50 percent

Observed manual results

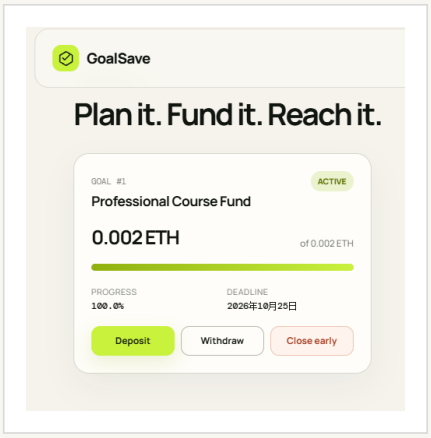
Step	Expected behavior	Observed result
Connect wallet	MetaMask account on Sepolia	Wallet and Sepolia indicator shown
Create goal	New goal and GoalCreated event	Goal 1 confirmed with target 0.002 ETH
First deposit	Balance becomes 0.001 ETH	Progress became 50 percent and event appeared
Early withdrawal	Normal withdrawal unavailable	Withdraw button remained disabled at 50 percent
Second deposit	Target reached	Balance became 0.002 ETH and Withdraw enabled
Withdraw	Owner receives full saved amount	Status became Withdrawn and balance became 0

Invalid input and unauthorized transaction paths were verified by automated tests. The final report does not claim separate browser screenshots for scenarios that were not manually captured.

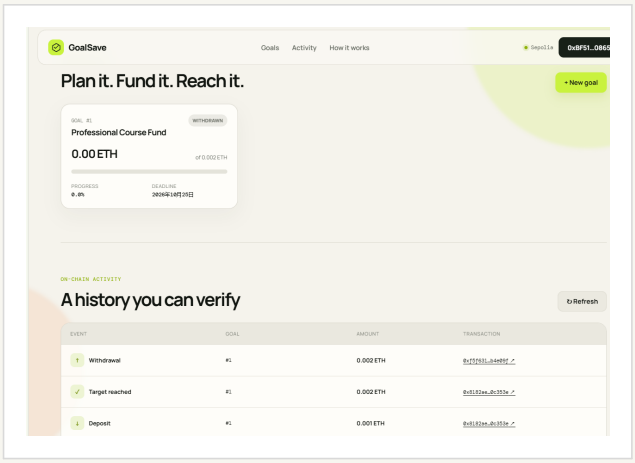
06 END TO END EVIDENCE

Target reached and withdrawal confirmed

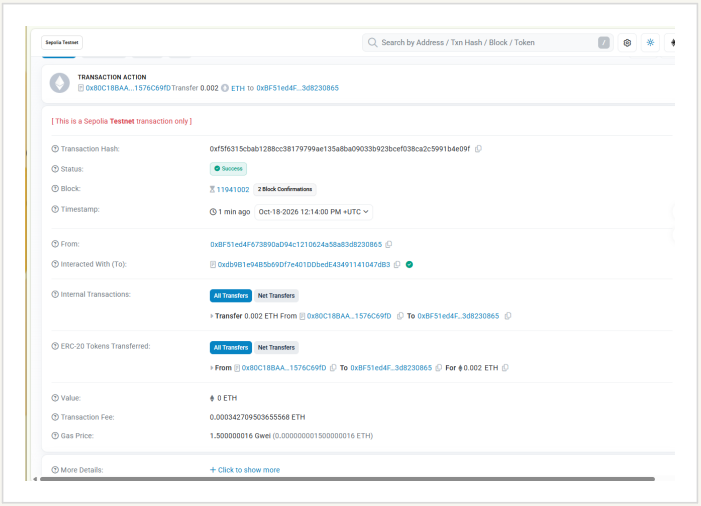
The second deposit raised the saved amount to the 0.002 ETH target. GoalSave then enabled normal withdrawal, returned the full balance, and recorded the terminal state and event on-chain.



Target reached at 100 percent and Withdraw enabled



Goal status changed to Withdrawn and the activity log recorded 0.002 ETH



Sepolia Etherscan receipt showing Success and the 0.002 ETH transfer from GoalSave to the wallet

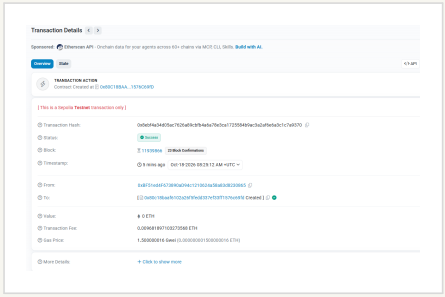
Withdrawal evidence	Value
Transaction	0xf5f6315cbab1288cc38179799ae135a8ba09033b923bcef038ca2c5991b4e09f
Block	11941002
Recipient wallet	0xBF51ed4F673890aD94c1210624a58a83d8230865
Amount returned	0.002 Sepolia ETH
Status	Success

07 DEPLOYMENT AND EVALUATION

Public submission record and evaluation

The same source repository supplies the contract, ABI, Flask application, frontend files, tests, Render blueprint, and deployment record. The web service and contract are separate deployments linked through public configuration.

Record	Final value
Student	HE HUZHENGXIONG G2509011E
GitHub	https://github.com/hehu6/GoalSave-SC6113-Individual-Assignment
Render	https://goalsave-sc6113.onrender.com
Contract	0x80c18baaf6102a26f5fedd337ef33ff1576c69fd
Deployment block	11939866
Deployment transaction	0x8ebf4a34d05ac7626a89cbfb4a6a78e3ca1725584b9ac3a2af6e6a3c1c7a9370



Contract creation on Sepolia block 11939866

Challenges encountered

Deployment required the Remix artifact, Sepolia address and block, Render variables, Flask configuration, and browser ABI to stay consistent. I stored the public values in deployment.json and exposed them through Flask. The interface also disables normal withdrawal while a goal is locked, while Solidity repeats the rule for callers that bypass the page.

Limitations

GoalSave is an unaudited testnet application. It supports native ETH only, relies on MetaMask and a Sepolia RPC, and scans events from the deployment block. A production history would require indexed event storage.

Future improvements

Future work would verify source code on Etherscan, add indexed history, support ERC 20 goals, improve contributor labels, and include accessibility and independent security testing.

Conclusion

GoalSave satisfies the SC6113 individual assignment requirements with an original financial use case and a complete deployed stack. Solidity enforces ownership and withdrawal rules, Flask provides validation and public configuration, and Sepolia records the full flow from goal creation through withdrawal.

References

[1] SC6113 Individual Assignment brief supplied by the module teaching team.
[2] Solidity documentation https://docs.soliditylang.org/
[3] MetaMask Wallet documentation https://docs.metamask.io/wallet/
[4] Sepolia Etherscan contract record https://sepolia.etherscan.io/address/0x80c18baaf6102a26f5fedd337ef33ff1576c69fd